

Guía Completa de Pruebas con Postman — cine-api

Tabla de Contenidos

1. [Introducción](#)
 2. [Configuración Inicial](#)
 3. [Pruebas de Endpoints de Películas](#)
 4. [Pruebas de Endpoints de Actores](#)
 5. [Flujo de Prueba Recomendado](#)
 6. [Verificación en MongoDB Atlas](#)
 7. [Troubleshooting](#)
-

1. Introducción

¿Qué es cine-api?

cine-api es una API REST desarrollada con **Node.js + Express** que se conecta a **MongoDB Atlas** para gestionar una base de datos de cine. Permite realizar operaciones **CRUD** (Crear, Leer, Actualizar, Eliminar) sobre dos recursos principales:

Recurso	Descripción
 Películas	Crear, listar, buscar, actualizar y eliminar películas
 Actores	Crear actores asociados a películas, listar y buscar por ID o por película

Requisitos Previos

Antes de comenzar, asegúrate de tener lo siguiente:

Requisito	Descripción
✓ Postman	Descargado e instalado desde postman.com/downloads (https://www.postman.com/downloads/)
✓ Node.js	Versión 18 o superior instalada
✓ Proyecto cine-api	Clonado y con dependencias instaladas (<code>npm install</code>)
✓ Archivo .env	Configurado con la cadena de conexión a MongoDB Atlas
✓ MongoDB Atlas	Clúster eva-u3-express activo y accesible

2. Configuración Inicial

Paso 1: Arrancar el servidor

Abre una terminal en la carpeta raíz del proyecto y ejecuta:

```
npm run dev
```

Deberías ver un mensaje similar a este en la consola:

```
🚀 Servidor corriendo en http://localhost:3000
✓ Conectado exitosamente a MongoDB Atlas
```

💡 **Tip:** Si usas `nodemon`, el servidor se reiniciará automáticamente al guardar cambios en el código.

Paso 2: Verificar conexión a MongoDB Atlas

Si ves el mensaje `✓ Conectado exitosamente a MongoDB Atlas`, significa que la API se conectó correctamente al clúster **eva-u3-express** y a la base de datos **cine_db**.

Si **no** ves ese mensaje, revisa la sección de [Troubleshooting](#).

Paso 3: Configurar Postman

1. Abre **Postman**
2. Crea una nueva **Colección** llamada `cine-api`
3. La URL base para todas las peticiones es:

```
http://localhost:3000
```

🔑 **Header obligatorio para peticiones con body (POST y PUT):**

Key	Value
Content-Type	application/json

3. 🎬 Pruebas de Endpoints de Películas

3.1 + Crear Película — POST /api/peliculas

Crea una nueva película en la base de datos.

Propiedad	Valor
Método	POST
URL	http://localhost:3000/api/peliculas
Header	Content-Type: application/json

📦 **Body (JSON):**

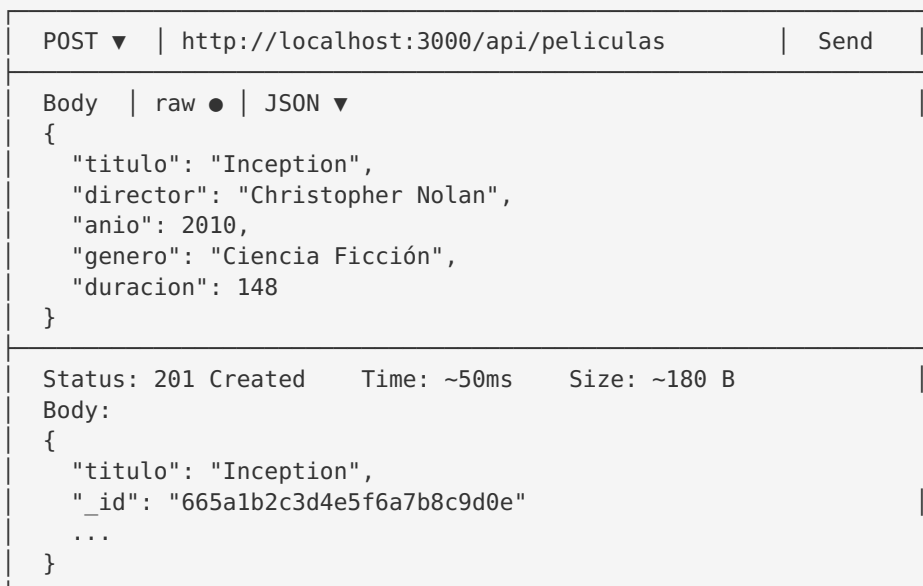
```
{
  "titulo": "Inception",
  "director": "Christopher Nolan",
  "anio": 2010,
  "genero": "Ciencia Ficción",
  "duracion": 148
}
```

✅ **Respuesta esperada — 201 Created :**

```
{
  "titulo": "Inception",
  "director": "Christopher Nolan",
  "anio": 2010,
  "genero": "Ciencia Ficción",
  "duracion": 148,
  "_id": "665a1b2c3d4e5f6a7b8c9d0e"
}
```

⚠️ **IMPORTANTE:** Copia y guarda el `_id` devuelto. Lo necesitarás para las pruebas de GET por ID, PUT y DELETE.

Qué debes ver en Postman:



✗ Caso de error — Body incompleto (400 Bad Request):

Si envías un body sin campos obligatorios, por ejemplo:

```

{
  "titulo": "Inception"
}

```

Respuesta:

```

{
  "mensaje": "Error de validación",
  "errores": [
    "El campo 'director' es requerido y debe ser un texto (string).",
    "El campo 'anio' es requerido.",
    "El campo 'genero' es requerido y debe ser un texto (string).",
    "El campo 'duracion' es requerido."
  ]
}

```

3.2 Listar Todas las Películas — GET /api/peliculas

Obtiene un array con todas las películas almacenadas.

Propiedad	Valor
Método	GET
URL	http://localhost:3000/api/peliculas
Header	Ninguno requerido
Body	Ninguno

✓ Respuesta esperada — 200 OK :

```
[
  {
    "_id": "665a1b2c3d4e5f6a7b8c9d0e",
    "titulo": "Inception",
    "director": "Christopher Nolan",
    "anio": 2010,
    "genero": "Ciencia Ficción",
    "duracion": 148
  }
]
```

💡 Si la colección está vacía, recibirás un array vacío: []

📷 Qué debes ver en Postman:

```
GET ▼ | http://localhost:3000/api/peliculas | Send |
Status: 200 OK    Time: ~30ms
Body:
[
  {
    "_id": "665a1b2c3d4e5f6a7b8c9d0e",
    "titulo": "Inception",
    "director": "Christopher Nolan",
    ...
  }
]
```

3.3 🔍 Buscar Película por ID — GET /api/peliculas/:id

Obtiene una película específica usando su `_id`.

Propiedad	Valor
Método	GET
URL	http://localhost:3000/api/peliculas/665a1b2c3d4e5f6a7b8c9d0e
Header	Ninguno requerido
Body	Ninguno

 Reemplaza 665a1b2c3d4e5f6a7b8c9d0e con el `_id` real obtenido al crear la película.

✓ Respuesta esperada — 200 OK :

```
{
  "_id": "665a1b2c3d4e5f6a7b8c9d0e",
  "titulo": "Inception",
  "director": "Christopher Nolan",
  "anio": 2010,
  "genero": "Ciencia Ficción",
  "duracion": 148
}
```

✗ Caso de error — ID no encontrado (404 Not Found):

Si usas un ID válido pero que no existe en la base de datos:

```
GET http://localhost:3000/api/peliculas/000000000000000000000000
```

Respuesta:

```
{
  "mensaje": "Película no encontrada con el ID proporcionado"
}
```

✗ Caso de error — ID con formato inválido (400 Bad Request):

```
GET http://localhost:3000/api/peliculas/esto-no-es-un-id
```

Respuesta:

```
{
  "mensaje": "El ID proporcionado no es un ObjectId válido de MongoDB"
}
```

Qué debes ver en Postman:

```

GET ▼ | http://localhost:3000/api/peliculas/<ID> | Send |
Status: 200 OK    Time: ~25ms
Body:
{
  "_id": "665a1b2c3d4e5f6a7b8c9d0e",
  "titulo": "Inception",
  "director": "Christopher Nolan",
  "anio": 2010,
  "genero": "Ciencia Ficción",
  "duracion": 148
}

```

3.4 Actualizar Película — PUT /api/peliculas/:id

Actualiza uno o más campos de una película existente.

Propiedad	Valor
Método	PUT
URL	http://localhost:3000/api/peliculas/ 665a1b2c3d4e5f6a7b8c9d0e
Header	Content-Type: application/json

 Reemplaza el ID en la URL con el `_id` real de la película a actualizar.

Body (JSON) — Actualizar género y duración:

```

{
  "genero": "Sci-Fi / Thriller",
  "duracion": 150
}

```

 **Nota:** Solo necesitas enviar los campos que deseas modificar. Los demás campos permanecerán sin cambios gracias al operador `$set` de MongoDB.

Respuesta esperada — 200 OK :

```

{
  "mensaje": "Película actualizada exitosamente",
  "modificados": 1
}

```

✗ Caso de error — Película no encontrada (404 Not Found):

```
{
  "mensaje": "Película no encontrada con el ID proporcionado"
}
```

✗ Caso de error — Body vacío (400 Bad Request):

Si envías un body vacío {} :

```
{
  "mensaje": "Debe enviar al menos un campo para actualizar"
}
```

📸 Qué debes ver en Postman:



```
PUT | http://localhost:3000/api/peliculas/<ID> | Send |
Body | raw • | JSON ▼
{
  "genero": "Sci-Fi / Thriller",
  "duracion": 150
}
Status: 200 OK   Time: ~40ms
Body:
{
  "mensaje": "Película actualizada exitosamente",
  "modificados": 1
}
```

3.5 🗑 Eliminar Película — DELETE /api/peliculas/:id

Elimina una película de la base de datos de forma permanente.

Propiedad	Valor
Método	DELETE
URL	http://localhost:3000/api/peliculas/ 665a1b2c3d4e5f6a7b8c9d0e
Header	Ninguno requerido
Body	Ninguno

🔄 Reemplaza el ID con el `_id` real de la película a eliminar.

✓ Respuesta esperada — 200 OK :

```
{
  "mensaje": "Película eliminada exitosamente"
}
```

✗ Caso de error — Película no encontrada (404 Not Found):

Si intentas eliminar una película que ya fue eliminada o no existe:

```
{
  "mensaje": "Película no encontrada con el ID proporcionado"
}
```

📷 Qué debes ver en Postman:

```
DELETE ▼ | http://localhost:3000/api/peliculas/<ID> | Send |
Status: 200 OK    Time: ~35ms
Body:
{
  "mensaje": "Película eliminada exitosamente"
}
```

⚠ **Nota:** Si la película tiene actores asociados, estos **no** se eliminan automáticamente. Los actores quedarán con un `peliculaId` que apunta a un documento que ya no existe.

4. 🧑‍🎭 Pruebas de Endpoints de Actores

4.1 ➕ Crear Actor — POST /api/actores

Crea un nuevo actor **asociado a una película existente**.

Propiedad	Valor
Método	POST
URL	http://localhost:3000/api/actores
Header	Content-Type: application/json

⚠ **MUY IMPORTANTE:** Para crear un actor debes incluir el campo `nombrePelicula` en el body. La API buscará la película por su título en la base de datos y asignará automáticamente el `peliculaId` correspondiente. **La película debe existir previamente.**

Body (JSON):

```
{
  "nombre": "Leonardo",
  "apellido": "DiCaprio",
  "edad": 49,
  "nacionalidad": "Estadounidense",
  "nombrePelícula": "Inception"
}
```

 **Observa:** No enviamos `películaId` directamente. Enviamos `nombrePelícula` y la API se encarga de buscar la película y obtener su `_id`. La búsqueda es **case-insensitive** (no distingue mayúsculas/minúsculas).

✓ Respuesta esperada — 201 Created :

```
{
  "nombre": "Leonardo",
  "apellido": "DiCaprio",
  "edad": 49,
  "nacionalidad": "Estadounidense",
  "películaId": "665a1b2c3d4e5f6a7b8c9d0e",
  "_id": "665b2c3d4e5f6a7b8c9d0f1a"
}
```

 Nota que en la respuesta aparece `películaId` (el `ObjectId` de la película) en lugar de `nombrePelícula`.

✗ Caso de error — Película no encontrada (400 Bad Request):

Si el nombre de la película no coincide con ninguna en la base de datos:

```
{
  "nombre": "Leonardo",
  "apellido": "DiCaprio",
  "edad": 49,
  "nacionalidad": "Estadounidense",
  "nombrePelícula": "Película Inexistente"
}
```

Respuesta:

```
{
  "mensaje": "La película 'Película Inexistente' no fue encontrada en la base de datos. Debe crear la película primero."
}
```

✗ Caso de error — Sin `nombrePelícula` (400 Bad Request):

Si no incluyes el campo `nombrePelícula`:

```
{
  "nombre": "Leonardo",
  "apellido": "DiCaprio",
  "edad": 49,
  "nacionalidad": "Estadounidense"
}
```

Respuesta:

```
{
  "mensaje": "El campo 'nombrePelicula' es requerido para asociar el actor a una película existente"
}
```

✗ Caso de error — Campos faltantes (400 Bad Request):

```
{
  "nombre": "Leonardo",
  "nombrePelicula": "Inception"
}
```

Respuesta:

```
{
  "mensaje": "Error de validación",
  "errores": [
    "El campo 'apellido' es requerido y debe ser un texto (string).",
    "El campo 'edad' es requerido.",
    "El campo 'nacionalidad' es requerido y debe ser un texto (string).",
  ]
}
```

Qué debes ver en Postman:



```

POST ▼ | http://localhost:3000/api/actores | Send |
Body | raw ● | JSON ▼
{
  "nombre": "Leonardo",
  "apellido": "DiCaprio",
  "edad": 49,
  "nacionalidad": "Estadounidense",
  "nombrePelicula": "Inception"
}

Status: 201 Created    Time: ~60ms
Body:
{
  "nombre": "Leonardo",
  "apellido": "DiCaprio",
  "peliculaId": "665a1b2c3d4e5f6a7b8c9d0e",
  "_id": "665b2c3d4e5f6a7b8c9d0f1a"
  ...
}

```

4.2 Listar Todos los Actores — GET /api/actores

Obtiene un array con todos los actores almacenados.

Propiedad	Valor
Método	GET
URL	http://localhost:3000/api/actores
Header	Ninguno requerido
Body	Ninguno

Respuesta esperada — 200 OK :



```

[
  {
    "_id": "665b2c3d4e5f6a7b8c9d0f1a",
    "nombre": "Leonardo",
    "apellido": "DiCaprio",
    "edad": 49,
    "nacionalidad": "Estadounidense",
    "peliculaId": "665a1b2c3d4e5f6a7b8c9d0e"
  }
]

```

Qué debes ver en Postman:

GET ▼ | http://localhost:3000/api/actores | Send |

Status: 200 OK Time: ~25ms
Body:

```
[
  {
    "_id": "665b2c3d4e5f6a7b8c9d0f1a",
    "nombre": "Leonardo",
    "apellido": "DiCaprio",
    ...
  }
]
```

4.3 🔍 Buscar Actor por ID — GET /api/actores/:id

Obtiene un actor específico usando su `_id`.

Propiedad	Valor
Método	GET
URL	http://localhost:3000/api/actores/ 665b2c3d4e5f6a7b8c9d0f1a
Header	Ninguno requerido
Body	Ninguno

 Reemplaza el ID con el `_id` real del actor obtenido al crearlo.

✓ Respuesta esperada — 200 OK :

```
{
  "_id": "665b2c3d4e5f6a7b8c9d0f1a",
  "nombre": "Leonardo",
  "apellido": "DiCaprio",
  "edad": 49,
  "nacionalidad": "Estadounidense",
  "peliculaId": "665a1b2c3d4e5f6a7b8c9d0e"
}
```

✗ Caso de error — Actor no encontrado (404 Not Found):

```
{
  "mensaje": "Actor no encontrado con el ID proporcionado"
}
```

✖ Caso de error — ID inválido (400 Bad Request):

```
{
  "mensaje": "El ID proporcionado no es un ObjectId válido de MongoDB"
}
```

🔍 Qué debes ver en Postman:

```
GET ▼ | http://localhost:3000/api/actores/<ID> | Send |
Status: 200 OK    Time: ~20ms
Body:
{
  "_id": "665b2c3d4e5f6a7b8c9d0f1a",
  "nombre": "Leonardo",
  "apellido": "DiCaprio",
  "edad": 49,
  "nacionalidad": "Estadounidense",
  "peliculaId": "665a1b2c3d4e5f6a7b8c9d0e"
}
```

4.4 🎬 Filtrar Actores por Película — GET /api/actores/pelicula/:peliculaId

Obtiene todos los actores asociados a una película específica.

Propiedad	Valor
Método	GET
URL	http://localhost:3000/api/actores/pelicula/665a1b2c3d4e5f6a7b8c9d0e
Header	Ninguno requerido
Body	Ninguno

🔄 Reemplaza el ID con el `_id` real de la **película** (no del actor).

✓ Respuesta esperada — 200 OK :

```
[
  {
    "_id": "665b2c3d4e5f6a7b8c9d0f1a",
    "nombre": "Leonardo",
    "apellido": "DiCaprio",
    "edad": 49,
    "nacionalidad": "Estadounidense",
    "peliculaId": "665a1b2c3d4e5f6a7b8c9d0e"
  }
]
```

💡 Si la película no tiene actores asociados, recibirás un array vacío: `[]`

✗ Caso de error — peliculaId inválido (400 Bad Request):

```
{
  "mensaje": "El peliculaId proporcionado no es un ObjectId válido de MongoDB"
}
```

📷 Qué debes ver en Postman:

```
GET ▼ | http://localhost:3000/api/actores/pelicula/<ID> | Send |
Status: 200 OK    Time: ~30ms
Body:
[
  {
    "_id": "665b2c3d4e5f6a7b8c9d0f1a",
    "nombre": "Leonardo",
    "apellido": "DiCaprio",
    "peliculaId": "665a1b2c3d4e5f6a7b8c9d0e"
    ...
  }
]
```

5. 🔄 Flujo de Prueba Recomendado

Orden sugerido paso a paso

Sigue este orden para probar toda la API de forma lógica:



Resumen de Casos de Prueba

Casos de Éxito

#	Endpoint	Código	Descripción
1	POST /api/peliculas	201	Crear película con datos completos y válidos
2	GET /api/peliculas	200	Listar todas las películas
3	GET /api/peliculas/:id	200	Buscar película con ID existente
4	PUT /api/peliculas/:id	200	Actualizar película existente
5	DELETE /api/peliculas/:id	200	Eliminar película existente
6	POST /api/actores	201	Crear actor con película existente
7	GET /api/actores	200	Listar todos los actores
8	GET /api/actores/:id	200	Buscar actor con ID existente
9	GET /api/actores/pelicula/:id	200	Filtrar actores por película

✖ Casos de Error

#	Endpoint	Código	Descripción
1	POST /api/peliculas	400	Body vacío o campos faltantes
2	POST /api/peliculas	400	Año fuera de rango (ej: anio: 1800)
3	POST /api/peliculas	400	Duración negativa (ej: duracion: -10)
4	GET /api/peliculas/:id	400	ID con formato inválido ("abc123")
5	GET /api/peliculas/:id	404	ID válido pero no existente
6	PUT /api/peliculas/:id	400	Body vacío {}
7	PUT /api/peliculas/:id	404	ID de película no existente
8	DELETE /api/peliculas/:id	404	ID de película no existente
9	POST /api/actores	400	Sin campo nombre-Pelicula
10	POST /api/actores	400	Película no encontrada por nombre
11	POST /api/actores	400	Campos del actor incompletos
12	GET /api/actores/:id	404	ID de actor no existente
13	GET /api/actores/pelicula/:id	400	peliculaId con formato inválido

6. Verificación en MongoDB Atlas

Después de realizar las pruebas en Postman, puedes verificar que los datos se guardaron correctamente en MongoDB Atlas.

Paso 1: Acceder a MongoDB Atlas

1. Ve a cloud.mongodb.com (<https://cloud.mongodb.com>) e inicia sesión
2. Selecciona el clúster **eva-u3-express**
3. Haz clic en **"Browse Collections"**

Paso 2: Navegar a la base de datos

1. En el panel izquierdo, selecciona la base de datos **cine_db**
2. Verás dos colecciones:
 - **películas**
 - **actores**

Paso 3: Qué verificar después de cada operación

Operación	Qué verificar en Atlas
POST película	Nuevo documento en películas con los campos enviados y un _id generado
PUT película	El documento actualizado con los nuevos valores en los campos modificados
DELETE película	El documento ya no aparece en la colección películas
POST actor	Nuevo documento en actores con peliculaId apuntando al _id de la película
GET (todos)	Comparar el array de Postman con los documentos visibles en Atlas

Ejemplo visual en Atlas:

```

cine_db
├── películas (1 documento)
│   └── {
│       _id: ObjectId("665a1b2c3d4e5f6a7b8c9d0e")
│       titulo: "Inception"
│       director: "Christopher Nolan"
│       anio: 2010
│       genero: "Ciencia Ficción"
│       duracion: 148
│   }
└── actores (1 documento)
    └── {
        _id: ObjectId("665b2c3d4e5f6a7b8c9d0f1a")
        nombre: "Leonardo"
        apellido: "DiCaprio"
        edad: 49
        nacionalidad: "Estadounidense"
        peliculaId: ObjectId("665a1b2c3d4e5f6a7b8c9d0e") ← Referencia a Inception
    }
  
```

 **Tip:** Puedes usar el botón **“Refresh”** en Atlas para ver los cambios más recientes después de cada operación en Postman.

7. Troubleshooting

Error: “No se pudo conectar a MongoDB Atlas”

Síntomas: El servidor arranca pero muestra un error de conexión a la base de datos.

Soluciones:

1. Verificar el archivo `.env` :

```
bash
# Asegúrate de que existe y contiene MONGODB_URI
cat .env
```

Debe tener algo similar a:

```
MONGODB_URI=mongodb+srv://usuario:contraseña@eva-u3-express.xxxxx.mongodb.net/cine_db
PORT=3000
```

2. Verificar IP en Atlas:

- Ve a **Network Access** en Atlas
- Asegúrate de que tu IP actual esté en la lista de IPs permitidas
- O agrega `0.0.0.0/0` para permitir acceso desde cualquier IP (solo para desarrollo)

3. Verificar credenciales:

- Ve a **Database Access** en Atlas
- Confirma que el usuario y contraseña del string de conexión son correctos

Error: “connect ECONNREFUSED 127.0.0.1:3000”

Síntomas: Postman muestra este error al enviar una petición.

Soluciones:

1. Verificar que el servidor está corriendo:

```
bash
# En la terminal donde ejecutaste npm run dev, deberías ver:
#  Servidor corriendo en http://localhost:3000
```

2. Reiniciar el servidor:

```
bash
# Detener con Ctrl + C y luego:
npm run dev
```

3. Verificar el puerto:

```
bash
# Comprobar si el puerto 3000 está en uso
lsof -i :3000
```

✗ Error: “Content-Type header is missing” o body no se recibe

Síntomas: La API recibe `undefined` o `{}` como body.

Soluciones:

1. En Postman, ve a la pestaña **Body**
2. Selecciona **raw**
3. En el dropdown, cambia de **Text** a **JSON**
4. Esto añade automáticamente el header `Content-Type: application/json`

✗ Error: “El campo ‘nombrePelicula’ es requerido...”

Síntomas: Al crear un actor, recibes error 400.

Solución: Asegúrate de incluir `"nombrePelicula"` en el body del POST de actores:

```
{
  "nombre": "Leonardo",
  "apellido": "DiCaprio",
  "edad": 49,
  "nacionalidad": "Estadounidense",
  "nombrePelicula": "Inception"  ✗ ¡No olvides este campo!
}
```

✗ Error: “La película ‘X’ no fue encontrada...”

Síntomas: Al crear un actor, la película no se encuentra.

Soluciones:

1. **Crear la película primero:** Ejecuta `POST /api/peliculas` antes de crear el actor
2. **Verificar el nombre exacto:** La búsqueda es case-insensitive pero debe coincidir completamente
 - ✓ `"nombrePelicula": "Inception"` → Funciona
 - ✓ `"nombrePelicula": "inception"` → Funciona (case-insensitive)
 - ✗ `"nombrePelicula": "Incep"` → No funciona (nombre incompleto)

✗ Error 500: “Error interno del servidor”

Síntomas: La API responde con código 500.

Soluciones:

1. **Revisar la consola del servidor** donde ejecutaste `npm run dev` — allí aparecerá el error detallado
2. **Verificar conexión a MongoDB:** Es posible que se haya perdido la conexión
3. **Reiniciar el servidor:** `Ctrl + C` y luego `npm run dev`

Checklist rápido antes de probar

Usa esta lista para verificar que todo está en orden:

- ☐  Servidor corriendo (`npm run dev`)
- ☐  Mensaje de conexión a MongoDB Atlas visible
- ☐  Postman abierto
- ☐  URL base correcta: `http://localhost:3000`
- ☐  Header `Content-Type: application/json` en peticiones POST y PUT
- ☐  Body en formato **raw** → **JSON** en Postman
- ☐  Películas creadas **antes** de crear actores

 **Nota final:** Todos los `_id` mostrados en esta guía son ejemplos. MongoDB generará IDs únicos diferentes cada vez que crees un nuevo documento. Siempre usa los IDs reales que obtengas de las respuestas de la API.

Guía creada para el proyecto cine-api — Taller 3: API REST con Node.js + Express + MongoDB Atlas 